

AWS Architecture and Database Service

Study Guide

AMARJIT SEAL
AI&DS, PIET
Parul University

1. AWS Well-Architected Framework

The AWS Well-Architected Framework is a set of best practices, guidelines, and design principles developed by AWS to help cloud architects build secure, high-performing, resilient, and efficient infrastructure for their applications and workloads.

1.1 Overview & Purpose

- Provides a consistent approach for customers and partners to evaluate architectures
- Identifies areas where workloads may not align with AWS best practices
- Gives guidance to implement improvements
- Assessed using the AWS Well-Architected Tool (free in the AWS Console)

1.2 The Six Pillars

The framework is organized around six pillars:

Pillar	Focus Area	Key Goal
1. Operational Excellence	Running and monitoring systems	Deliver business value and continuously improve
2. Security	Protecting information and systems	Protect data, systems, and assets
3. Reliability	Workload recovery and availability	Recover from failures, meet demands dynamically
4. Performance Efficiency	Using IT resources efficiently	Use compute resources efficiently as demands change
5. Cost Optimization	Avoiding unnecessary costs	Run systems at lowest price point
6. Sustainability	Environmental impacts	Reduce energy consumption & carbon footprint

1.3 Pillar 1 — Operational Excellence

Design Principles:

- Perform operations as code — treat infrastructure the same as application code
- Make frequent, small, reversible changes — reduces risk and allows easy rollback
- Refine operations procedures frequently — evolve procedures as workloads evolve
- Anticipate failure — run game days, test failure scenarios, validate recovery
- Learn from all operational failures — share lessons learned across teams

1.4 Pillar 2 — Security

Design Principles:

- Implement a strong identity foundation — use least-privilege access, centralize identity management
- Enable traceability — monitor, alert, and audit all actions and changes
- Apply security at all layers — defence-in-depth approach
- Automate security best practices — use software-based security mechanisms
- Protect data in transit and at rest — use encryption and tokenisation
- Keep people away from data — reduce or eliminate direct access to data
- Prepare for security events — incident management and investigation policies

1.5 Pillar 3 — Reliability

Design Principles:

- Automatically recover from failure — monitor KPIs and trigger automation on thresholds
- Test recovery procedures — use automation to simulate and test failure scenarios
- Scale horizontally to increase availability — replace one large resource with multiple smaller ones
- Stop guessing capacity — monitor demand, add/remove resources automatically
- Manage change through automation — use IaC for infrastructure changes

1.6 Pillar 4 — Performance Efficiency

Design Principles:

- Democratize advanced technologies — allow engineering teams to focus on product development
- Go global in minutes — deploy workloads in multiple AWS Regions for low latency

- Use serverless architectures — remove operational burden of managing servers
- Experiment more often — use virtual resources and run comparative tests
- Consider mechanical sympathy — understand how services match usage patterns

1.7 Pillar 5 — Cost Optimization

Design Principles:

- Implement cloud financial management — build capability through knowledge, programs, and resources
- Adopt a consumption model — pay only for resources you consume
- Measure overall efficiency — measure business output vs cost of delivering it
- Stop spending money on undifferentiated heavy lifting — AWS manages data centres
- Analyze and attribute expenditure — accurately identify cost and usage, ROI measurement

1.8 Pillar 6 — Sustainability

Design Principles:

- Understand your impact — measure the impact of cloud workloads
- Establish sustainability goals — set long-term goals for improvements
- Maximize utilization — right-size workloads and use efficient hardware
- Anticipate and adopt new efficient hardware and software offerings
- Use managed services — shift demand to reduce customer infrastructure footprint
- Reduce downstream impact — reduce energy or resources required for use

1.9 General AWS Cloud Design Principles (Framework-Wide)

Principle	Description
Stop guessing capacity needs	Use as-needed scaling; avoid over-provisioning
Test systems at production scale	Create full production-scale test environments on demand
Automate with architectural experimentation	Automation supports creation and replication at lower cost
Allow evolutionary architectures	Design changes as a regular occurrence, not one-time events

Principle	Description
Drive architectures using data	Collect data on workload behaviour and use for decisions
Improve through game days	Schedule and simulate events to understand areas for improvement
Loosely couple components	Reduce interdependencies; failure in one component doesn't cascade

Exam Tip: Each pillar has specific design principles — memorise which principle belongs to which pillar. Most common MCQ: 'Automatically recover from failure' → Reliability; 'Make frequent, small, reversible changes' → Operational Excellence; 'Go global in minutes' → Performance Efficiency; 'Adopt a consumption model' → Cost Optimization.

2. Amazon RDS (Relational Database Service)

Amazon RDS is a fully managed, distributed relational database service that makes it easy to set up, operate, and scale a relational database in the cloud. It automates time-consuming administration tasks such as hardware provisioning, database setup, patching, and backups.

2.1 Supported Database Engines

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- Microsoft SQL Server
- Amazon Aurora (MySQL and PostgreSQL compatible)

2.2 Key Features of Amazon RDS

Feature	Details
Automated Backups	Enabled by default; point-in-time recovery; retained 1–35 days
DB Snapshots	User-initiated; persisted until explicitly deleted

Feature	Details
Multi-AZ Deployment	Synchronous standby replica in different AZ for high availability and failover
Read Replicas	Asynchronous replication for read-heavy workloads (up to 5 replicas for MySQL/PostgreSQL)
Automated Patching	AWS manages OS and DB engine patches automatically
Encryption	At-rest using AES-256 (AWS KMS); in-transit using SSL/TLS
Monitoring	Amazon CloudWatch metrics, Enhanced Monitoring, Performance Insights
Scaling	Vertical scaling (instance class) and storage auto-scaling
Security	VPC isolation, Security Groups, IAM authentication
Deletion Protection	Prevents accidental deletion of DB instances

2.3 Multi-AZ vs. Read Replicas

Feature	Multi-AZ	Read Replica
Purpose	High Availability / Disaster Recovery	Performance / Scalability
Replication	Synchronous	Asynchronous
Endpoint	Same endpoint (automatic failover)	Separate endpoint
Read Capable?	No (standby only)	Yes (read-only)
Cross-Region?	No	Yes (cross-region replicas)

Feature	Multi-AZ	Read Replica
Failover Time	~1–2 minutes (automatic)	Manual promotion required

2.4 RDS Storage Types

- General Purpose SSD (gp2/gp3) — balanced price/performance, default for most workloads
- Provisioned IOPS SSD (io1/io2) — high-performance for I/O-intensive workloads
- Magnetic — legacy, low-cost, not recommended for new deployments

2.5 MySQL on Amazon RDS

MySQL is the most popular open-source relational database in the world. On Amazon RDS, MySQL offers full compatibility with standard MySQL features.

Key Characteristics:

- Supported versions: MySQL 5.7, 8.0
- Storage engines: InnoDB (default and recommended), MyISAM (limited support)
- Automated backups, point-in-time recovery (PITR)
- Up to 5 Read Replicas (can be cross-region)
- Multi-AZ for high availability
- Parameter groups to customize engine configuration
- Option groups for additional features
- Encryption using AWS KMS (must be enabled at creation)
- Maximum storage: 64 TB (gp3/io1)

Use Cases:

- Web applications, content management systems (WordPress, Drupal)
- E-commerce applications
- OLTP (Online Transaction Processing)

2.6 PostgreSQL on Amazon RDS

PostgreSQL is an advanced, open-source object-relational database system known for its robustness and standards compliance.

Key Characteristics:

- Supported versions: PostgreSQL 12, 13, 14, 15, 16

- Supports JSON/JSONB for semi-structured data (hybrid SQL + NoSQL)
- Advanced data types: arrays, hstore, UUID, geometric types
- Full-text search, geospatial extensions (PostGIS)
- Up to 5 Read Replicas; logical replication supported
- pgBouncer for connection pooling
- Compatible with AWS Aurora PostgreSQL

Use Cases:

- Financial systems requiring complex queries and data integrity
- Geospatial applications
- Scientific and research databases
- Applications requiring advanced SQL features

2.7 Microsoft SQL Server on Amazon RDS

Amazon RDS for SQL Server provides a managed environment for running Microsoft SQL Server in the AWS Cloud.

Supported Editions:

- Express, Web, Standard, Enterprise

Key Characteristics:

- Supports Windows and SQL Server Authentication
- Backup/Restore to Amazon S3
- Supports Active Directory integration (Windows Authentication)
- Multi-AZ support using SQL Server Database Mirroring or Always On
- Licensing: License Included (AWS manages) or Bring Your Own License (BYOL)
- Transparent Data Encryption (TDE) via option groups
- Native backups to S3 using the SQLSERVER_BACKUP_RESTORE option

Limitations vs. On-Premises:

- No support for cross-region Read Replicas (unlike MySQL/PostgreSQL)
- Limited instances per region
- Some features restricted in managed mode

3. Amazon Aurora

Amazon Aurora is a MySQL and PostgreSQL-compatible relational database engine built for the cloud. Aurora combines the speed and availability of high-end commercial databases with the simplicity and cost-effectiveness of open-source databases.

3.1 Key Facts

- Proprietary AWS-built database engine — NOT open source
- MySQL compatible: 5x faster than standard MySQL
- PostgreSQL compatible: 3x faster than standard PostgreSQL
- Automatically grows storage in 10 GB increments up to 128 TB
- Aurora storage replicates 6 copies across 3 Availability Zones automatically
- Self-healing: data blocks and disks are continuously scanned and repaired
- Up to 15 Aurora Replicas (vs. 5 for standard RDS)

3.2 Aurora Architecture

Cluster Volume:

- Shared distributed storage layer — separate from compute
- Data is replicated 6 times across 3 AZs (2 copies per AZ)
- Can tolerate up to 2 copy failures for writes and 3 for reads

Endpoints:

- Cluster (Writer) Endpoint — connects to the primary instance for read/write
- Reader Endpoint — load balances across all Read Replicas
- Instance Endpoint — connects to a specific DB instance
- Custom Endpoint — connects to a subset of instances

3.3 Aurora Features

Feature	Description
Aurora Serverless v1/v2	Auto-scales compute capacity based on workload; ideal for intermittent use
Aurora Global Database	Single primary in one region, up to 5 secondary read-only regions; <1 second replication
Aurora Multi-Master	Multiple write nodes for continuous availability (MySQL only)

Feature	Description
Backtrack	Rewind database to a previous point in time without restoring from backup (MySQL only)
Parallel Query	Pushes query processing down to storage layer for faster analytics
Database Cloning	Creates a clone quickly using copy-on-write protocol
Zero-ETL with Redshift	Near real-time data integration with Amazon Redshift

3.4 Aurora vs. Standard RDS Comparison

Attribute	Amazon Aurora	Standard RDS (MySQL/PostgreSQL)
Storage	Auto-grows up to 128 TB	Manual allocation up to 64 TB
Replicas	Up to 15	Up to 5
Failover	~30 seconds	~1–2 minutes
Replication	6 copies, 3 AZs	1 standby (Multi-AZ)
Cost	Higher (~20% over RDS)	Lower
Performance	5x MySQL / 3x PostgreSQL	Standard engine performance
Backtrack	Yes (MySQL)	No
Serverless	Yes (v1 & v2)	No

4. Amazon DynamoDB

Amazon DynamoDB is a fully managed, serverless, NoSQL database service that provides key-value and document data structures. It is designed to run high-performance applications at any scale.

4.1 Core Characteristics

- Fully managed — no servers to provision, patch, or manage
- Serverless — scales automatically to handle any level of traffic
- NoSQL database (Key-Value and Document model)
- Single-digit millisecond performance at any scale
- Multi-Region, multi-active with DynamoDB Global Tables
- Built-in security, backup and restore, and in-memory caching
- Event-driven computing with AWS Lambda integration

4.2 DynamoDB Core Concepts

Concept	Description
Table	Collection of items (equivalent to a table in relational DBs)
Item	A single data record in a table (equivalent to a row)
Attribute	A fundamental data element in an item (equivalent to a column)
Partition Key (PK)	Primary key; DynamoDB hashes this to determine physical partition
Sort Key (SK)	Optional; combined with PK to form a composite primary key
Global Secondary Index (GSI)	Alternate partition key + optional sort key; spans all partitions
Local Secondary Index (LSI)	Same partition key as table, different sort key; must be defined at table creation

4.3 Capacity Modes

Provisioned Mode (Default):

- You specify Read Capacity Units (RCUs) and Write Capacity Units (WCUs)
- 1 RCU = 1 strongly consistent read/sec for items up to 4 KB
- 1 RCU = 2 eventually consistent reads/sec for items up to 4 KB
- 1 WCU = 1 write/sec for items up to 1 KB
- Auto-scaling available

On-Demand Mode:

- Pay-per-request; no capacity planning needed
- Instantly accommodates workloads as they ramp up
- Ideal for unpredictable or new workloads

4.4 DynamoDB Advanced Features

Feature	Description
DynamoDB Streams	Captures item-level changes in real time; enables event-driven architectures
DynamoDB Accelerator (DAX)	In-memory cache; reduces response times from milliseconds to microseconds
Global Tables	Multi-region, multi-active replication; active-active across regions
Point-in-Time Recovery (PITR)	Enables restoring table to any second in the past 35 days
Time-to-Live (TTL)	Automatically deletes items after a specified timestamp
Transactions	ACID transactions across multiple items/tables
Export to S3	Export table data to S3 without consuming capacity
PartiQL	SQL-compatible query language for DynamoDB

4.5 When to Use DynamoDB vs. RDS

Use DynamoDB When...	Use RDS When...
Need massive scale (millions of TPS)	Need complex SQL queries & joins
Schema is flexible / changes frequently	Schema is well-defined and structured
Need ultra-low latency (sub-millisecond with DAX)	Need ACID transactions across many tables

Use DynamoDB When...	Use RDS When...
Serverless architecture preferred	Applications with complex reporting needs
Session management, gaming leaderboards, IoT	ERP, CRM, financial systems

5. Amazon Redshift

Amazon Redshift is a fully managed, petabyte-scale data warehouse service in the cloud. It is designed for Online Analytical Processing (OLAP) — analyzing large amounts of data quickly for business intelligence (BI) applications.

5.1 Core Characteristics

- Based on PostgreSQL — uses SQL to analyze structured and semi-structured data
- Columnar storage — stores data by column instead of row for faster analytical queries
- Massively Parallel Processing (MPP) — distributes query execution across multiple nodes
- Can scale from a few hundred gigabytes to petabytes
- Integrates with BI tools: Tableau, QuickSight, PowerBI, Looker
- Data loaded from S3, DynamoDB, DMS, and RDS Read Replicas

5.2 Redshift Architecture

Cluster:

- A Redshift cluster = one or more compute nodes
- Leader Node — manages client communication, query planning, and results aggregation
- Compute Nodes — store data and perform queries
- Node Slices — each compute node is divided into slices that process queries in parallel

Node Types:

- RA3 Nodes — separate compute and storage; storage on Amazon S3 (managed)
- DC2 Nodes — compute-intensive; fast local SSD storage
- DS2 Nodes — dense storage; large amounts of magnetic disk storage (legacy)

5.3 Redshift Key Features

Feature	Description
Redshift Spectrum	Query data directly in Amazon S3 without loading into Redshift
Redshift Serverless	Automatic provisioning and scaling; pay only when queries run
Materialized Views	Precomputed result sets for faster repeated query performance
Concurrency Scaling	Automatically adds cluster capacity to handle concurrent queries
AQUA (Advanced Query Accelerator)	Hardware-accelerated cache for faster query performance
Zero-ETL with Aurora	Near real-time data replication from Aurora to Redshift
Data Sharing	Share live data across clusters without copying
ML with Redshift	Create, train, and deploy ML models using SQL (Redshift ML)

5.4 Redshift vs. RDS vs. DynamoDB — Summary

Criteria	Amazon RDS	Amazon DynamoDB	Amazon Redshift
Type	Relational (OLTP)	NoSQL (Key-Value/Document)	Data Warehouse (OLAP)
Data Model	Structured (SQL)	Unstructured / Semi-structured	Structured (columnar SQL)
Use Case	Transactions, web apps	Real-time, gaming, IoT, mobile	Analytics, BI, reporting

Criteria	Amazon RDS	Amazon DynamoDB	Amazon Redshift
Scale	Vertical + Read Replicas	Horizontal (massive scale)	Horizontal (petabyte)
Query Language	SQL	PartiQL / DynamoDB API	SQL (PostgreSQL compatible)
Latency	Milliseconds	Single-digit milliseconds	Seconds (complex queries)
Storage Model	Row-based	Key-Value / Document	Columnar
Serverless Option	No (Aurora Serverless)	Yes	Yes (Redshift Serverless)

6. Launching an Amazon RDS DB Instance

Step-by-step guide to creating an RDS DB instance using the AWS Management Console.

6.1 Prerequisites

- An active AWS account
- IAM user/role with RDS and VPC permissions
- VPC with at least 2 subnets in different Availability Zones
- Security group configured to allow inbound DB traffic (e.g., port 3306 for MySQL)

6.2 Step-by-Step: Create RDS DB Instance (Console)

Step 1: Open the RDS Console

- Sign in to the AWS Management Console
- Navigate to Services → Database → RDS
- Select the desired AWS Region in the top-right corner

Step 2: Start Creating a Database

- In the navigation pane, click 'Databases'
- Click 'Create database' button

Step 3: Choose a Database Creation Method

- Standard Create — full control over all configuration settings
- Easy Create — uses recommended best-practice configurations

Step 4: Choose Engine Options

- Select the engine: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, or Aurora
- Select the specific version number

Step 5: Choose a Template

- Production — enables Multi-AZ and Provisioned IOPS for high availability
- Dev/Test — for development environments outside production
- Free tier — single AZ, db.t2.micro/t3.micro/t4g.micro (750 hrs/month free)

Step 6: Configure Settings

- DB Instance Identifier — unique name within your AWS account and region
- Master Username — admin user for the DB
- Master Password — set or auto-generate

Step 7: Choose DB Instance Class

- Standard classes (m series) — balance of compute/memory/network
- Memory-optimized (r/x series) — for memory-intensive workloads
- Burstable (t series) — cost-effective for dev/test (free tier eligible)

Step 8: Configure Storage

- Storage type: gp3 (General Purpose), io1 (Provisioned IOPS), or Magnetic
- Allocated storage: specify initial storage in GB
- Enable Storage Autoscaling: automatically scales up when storage is low

Step 9: Availability & Durability

- Multi-AZ DB Instance — creates a standby replica in a different AZ
- Single AZ — lower cost, lower availability

Step 10: Connectivity

- VPC — select the VPC where the instance will be launched
- DB Subnet Group — defines subnets in which the instance can be placed
- Public Access — Yes (public IP assigned) or No (VPC only; best practice)

- VPC Security Group — controls inbound/outbound traffic
- Port — default: MySQL=3306, PostgreSQL=5432, SQL Server=1433

Step 11: Database Authentication

- Password Authentication — uses master username and password
- IAM Authentication — uses IAM tokens (no password needed)
- Kerberos Authentication — for Active Directory integration

Step 12: Additional Configuration

- Initial database name — creates a default schema on startup
- DB Parameter Group — controls engine settings (e.g., max_connections)
- Option Group — enables additional features (e.g., TDE for SQL Server)
- Backup retention period — 0 (disable) to 35 days
- Backup window — preferred time window for automated backups
- Enhanced Monitoring — OS-level metrics at 1, 5, 10, 15, 30, 60-second intervals
- Performance Insights — database performance analysis
- Deletion Protection — prevents accidental database deletion

Step 13: Create the Database

- Review all settings and click 'Create database'
- Status shows 'Creating' — wait for 'Available' status (typically 5–10 minutes)
- Note the Endpoint and Port — used to connect your application

6.3 Connecting to an RDS Instance

Engine	Default Port	Connection Tool
MySQL	3306	MySQL Workbench, MySQL CLI, DBeaver
PostgreSQL	5432	pgAdmin, psql CLI, DBeaver
SQL Server	1433	SQL Server Management Studio (SSMS), Azure Data Studio
Oracle	1521	SQL Developer, SQL*Plus
MariaDB	3306	HeidiSQL, MySQL Workbench, CLI

6.4 Create RDS via AWS CLI (Example — SQL Server)

Bash

```
aws rds create-db-instance \  
--engine sqlserver-se \  
--db-instance-identifier my-sql-server \  
--db-instance-class db.t3.large \  
--allocated-storage 250 \  
--master-username admin \  
--manage-master-user-password \  
--backup-retention-period 7
```

7. Important Exam Questions & Answers

The following questions are compiled from university previous year papers, AWS certification exams, and standard academic assessments on these topics.

7.1 AWS Well-Architected Framework Questions

Q1. What is the AWS Well-Architected Framework? What are its six pillars?

Ans: The AWS Well-Architected Framework is a set of best practices for building secure, high-performing, resilient, and efficient infrastructure on AWS. The six pillars are: (1) Operational Excellence, (2) Security, (3) Reliability, (4) Performance Efficiency, (5) Cost Optimization, and (6) Sustainability. Each pillar provides a set of design principles and best practices.

Q2. Which design principle belongs to the Operational Excellence pillar of the AWS Well-Architected Framework? (a) Go global in minutes (b) Anticipate failure (c) Automatically recover from failure (d) Adopt a consumption model

Ans: (b) Anticipate failure. Operational Excellence principles include: Perform operations as code, Make frequent/small/reversible changes, Refine operations frequently, Anticipate failure, and Learn from all operational failures.

Q3. What is the purpose of Multi-AZ deployment in Amazon RDS? How does it differ from Read Replicas?

Ans: Multi-AZ creates a synchronous standby replica in a different Availability Zone for high availability and automatic failover (disaster recovery). Read Replicas are asynchronous copies used to scale read traffic. Multi-AZ is for HA/DR; Read Replicas are for performance.

Q4. Which design principle of the Reliability pillar states 'Automatically recover from failure'? Explain with an example.

Ans: The Reliability pillar includes the principle 'Automatically recover from failure.' This means monitoring key performance indicators (KPIs) and triggering automation when a

threshold is breached. Example: Using CloudWatch alarms to trigger Auto Scaling or AWS Lambda to restart failed instances automatically.

Q5. Explain the difference between 'Stop guessing capacity needs' and 'Test systems at production scale' in the Well-Architected Framework.

Ans: 'Stop guessing capacity needs' means using cloud elasticity to scale automatically rather than over-provisioning. 'Test systems at production scale' means creating a full production-scale test environment on demand to identify bottlenecks before they occur in production — then decommissioning after testing.

Q6. Which design principle is a GENERAL AWS Cloud principle described in the Well-Architected Framework? (a) Monolithic architecture (b) Test systems at production scale (c) Provision excess capacity (d) Make one-time architectural decisions

Ans: (b) Test systems at production scale. The general principles are: Stop guessing capacity, Test at production scale, Automate, Allow evolutionary architectures, Drive architectures using data, Improve through game days.

7.2 Amazon RDS Questions

Q7. What is Amazon RDS? List the database engines supported by Amazon RDS.

Ans: Amazon RDS (Relational Database Service) is a fully managed relational database service that automates hardware provisioning, patching, backups, and scaling. Supported engines: MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server, and Amazon Aurora (MySQL and PostgreSQL compatible).

Q8. True or False: Amazon RDS automatically patches the database software and backs up your database, storing the backups for a user-defined retention period and enabling point-in-time recovery.

Ans: TRUE. RDS automatically applies patches during a user-defined maintenance window and performs automated backups with a configurable retention period (1–35 days), enabling point-in-time recovery.

Q9. What are the key differences between MySQL, PostgreSQL, and SQL Server on Amazon RDS?

Ans: MySQL: Popular open-source RDBMS, best for web apps (WordPress, e-commerce), 5 read replicas, InnoDB storage engine. PostgreSQL: Advanced open-source RDBMS, supports JSON/JSONB, advanced data types, PostGIS for geospatial, better standards compliance. SQL Server: Microsoft proprietary, supports Windows

Authentication, TDE, SSMS integration, requires Microsoft licensing (License Included or BYOL).

Q10. A company needs its database to automatically failover in case the primary database becomes unavailable. Which RDS feature should they use?

Ans: Multi-AZ (Multi-Availability Zone) deployment. It creates a synchronous standby replica in a different AZ. In case of primary failure, RDS automatically updates the DNS endpoint to point to the standby, achieving failover in approximately 1–2 minutes.

Q11. What is the maximum number of Read Replicas you can create for a MySQL RDS instance? For Aurora?

Ans: Standard RDS MySQL/PostgreSQL: up to 5 Read Replicas. Amazon Aurora: up to 15 Aurora Replicas.

Q12. Describe the steps to launch an Amazon RDS DB instance using the AWS Management Console.

Ans: 1) Sign into AWS Console → open RDS service. 2) Choose 'Create database.' 3) Select Standard or Easy Create. 4) Select engine (MySQL/PostgreSQL/etc.). 5) Choose template (Production/Dev/Free Tier). 6) Configure DB identifier, username, and password. 7) Select DB instance class. 8) Configure storage type and size. 9) Set availability (Multi-AZ or Single AZ). 10) Configure VPC, subnet group, security groups, and port. 11) Set authentication method. 12) Configure backups, monitoring, and deletion protection. 13) Click 'Create database.'

7.3 Amazon Aurora Questions

Q13. What makes Amazon Aurora different from standard Amazon RDS? What are its performance claims?

Ans: Aurora is AWS's proprietary cloud-native relational database compatible with MySQL and PostgreSQL. Key differences: 5x faster than MySQL, 3x faster than PostgreSQL, auto-grows storage up to 128 TB in 10 GB increments, replicates data 6 times across 3 AZs, supports up to 15 replicas, faster failover (~30 seconds), supports Backtrack, Aurora Serverless, and Global Database.

Q14. What is Aurora Serverless? When would you use it?

Ans: Aurora Serverless is an on-demand, auto-scaling configuration for Amazon Aurora. It automatically starts up, shuts down, and scales capacity based on application needs. Use cases: infrequently used applications, new applications with unpredictable

workloads, development/test environments, and applications with variable or intermittent traffic.

Q15. A company needs a relational database that automatically replicates data across multiple AWS regions with less than 1 second lag. Which AWS service should they use?

Ans: Amazon Aurora Global Database. It spans multiple AWS regions with a single primary region and up to 5 secondary read-only regions. Replication lag is typically under 1 second. In case of regional failure, a secondary region can be promoted to primary in under 1 minute.

Q16. An ecommerce application is expected to handle large amounts of structured data and requires a highly available database with fast read performance. Which would be the ideal database on AWS?

Ans: Amazon Aurora. It provides high availability (6 copies, 3 AZs), up to 15 Aurora Replicas for fast reads, auto-scaling storage, and better performance than standard RDS. It is ideal for read-intensive, structured-data enterprise applications.

7.4 Amazon DynamoDB Questions

Q17. What is Amazon DynamoDB? How does it differ from Amazon RDS?

Ans: DynamoDB is a fully managed, serverless NoSQL key-value and document database. Differences from RDS: DynamoDB is NoSQL (no fixed schema), scales horizontally to any scale, offers single-digit millisecond latency, is serverless, and best for flexible/unstructured data. RDS is a relational (SQL) database with a fixed schema, ACID compliance, and best for structured/transactional data.

Q18. If you are developing an application that requires a database with extremely fast performance, fast scalability, and flexibility in the database schema, which AWS service would you consider?

Ans: Amazon DynamoDB. It provides sub-millisecond latency, horizontal auto-scaling to any level of traffic, and a flexible schema (NoSQL) that allows different attributes per item. It is ideal for gaming, real-time applications, mobile apps, and IoT.

Q19. What is DynamoDB Accelerator (DAX)? When should it be used?

Ans: DAX (DynamoDB Accelerator) is a fully managed, highly available, in-memory cache for DynamoDB. It reduces read response times from milliseconds to microseconds. Use it when applications require the fastest possible read performance

and cannot tolerate even millisecond latency — e.g., gaming leaderboards, real-time bidding, financial trading.

Q20. Explain DynamoDB Global Tables and their use case.

Ans: DynamoDB Global Tables provide multi-region, multi-active replication. Any changes written to a table in one region are automatically replicated to all other regions within seconds. Use case: globally distributed applications that need low-latency read and write access in multiple regions, such as global e-commerce platforms or gaming applications.

Q21. What is the difference between a Partition Key and a Sort Key in DynamoDB? What is a composite primary key?

Ans: Partition Key (Hash Key): A single attribute used as the primary key. DynamoDB hashes this value to determine the partition where the item is stored. Sort Key (Range Key): An optional second attribute. Combined with the Partition Key, it forms a Composite Primary Key. Items with the same partition key are stored together, sorted by sort key — enabling range queries.

Q22. What is Amazon DynamoDB Streams? Explain its use with AWS Lambda.

Ans: DynamoDB Streams captures a time-ordered sequence of item-level modifications in a DynamoDB table. Each modification (insert/update/delete) creates a stream record. When integrated with AWS Lambda, any change to a DynamoDB table can automatically trigger a Lambda function — enabling event-driven architectures such as sending notifications, updating caches, or replicating data.

7.5 Amazon Redshift Questions

Q23. What is Amazon Redshift? How is it different from Amazon RDS?

Ans: Amazon Redshift is a fully managed cloud data warehouse for OLAP (Online Analytical Processing). It uses columnar storage and MPP (Massively Parallel Processing). Difference from RDS: RDS is for OLTP (transactional, real-time operations), while Redshift is for OLAP (analytical queries, reporting, BI). Redshift handles petabyte-scale data; RDS handles transactional workloads.

Q24. What is Massively Parallel Processing (MPP) in Amazon Redshift?

Ans: MPP means Redshift distributes and parallelizes query execution across all available compute nodes. When a query is submitted, the leader node compiles the query and distributes execution code to compute nodes, which run queries in parallel

across slices. This enables Redshift to process complex analytical queries on petabytes of data very quickly.

Q25. What is the difference between columnar and row-based storage? Why does Redshift use columnar storage?

Ans: Row-based storage (RDS, MySQL): entire rows are stored together — efficient for transactional operations (insert/update/delete). Columnar storage (Redshift): each column is stored separately — efficient for analytical queries that select only specific columns from large datasets. Redshift uses columnar storage because analytical queries typically aggregate specific columns across millions of rows, requiring far less I/O than row-based storage.

Q26. What is Redshift Spectrum? How is it different from standard Redshift queries?

Ans: Redshift Spectrum allows querying data stored in Amazon S3 directly without loading it into Redshift. Standard Redshift queries run on data stored in Redshift cluster nodes. Spectrum extends query capability to exabytes of data in S3 — useful for data lake architectures where historical/cold data stays in S3 while hot data is in Redshift.

Q27. Which AWS database service is best suited for each scenario? (a) E-commerce transactional system (b) Gaming leaderboard with millions of users (c) Business Intelligence reporting on 100 TB of data (d) A globally available social media feed

Ans: (a) Amazon RDS / Aurora — structured data, ACID transactions, SQL queries. (b) Amazon DynamoDB — serverless, single-digit ms latency, massive scale. (c) Amazon Redshift — data warehouse, columnar storage, complex analytical queries. (d) Amazon DynamoDB Global Tables — multi-region, multi-active, low-latency global access.

7.6 Mixed / Scenario Questions

Q28. A company uses an Amazon RDS MySQL instance. To improve read performance without affecting the primary database, what should they implement?

Ans: Read Replicas. Up to 5 Read Replicas can be created for RDS MySQL. Read traffic is directed to replicas using their individual endpoints, reducing load on the primary instance. Read Replicas use asynchronous replication and can also be promoted to standalone DB instances if needed.

Q29. What are the key considerations when choosing a database type on AWS?

Ans: Key considerations: (1) Data structure — is data structured (SQL) or unstructured (NoSQL)? (2) Query patterns — simple key-value lookups (DynamoDB) or complex joins (RDS/Aurora)? (3) Scale — millions of TPS (DynamoDB) or enterprise scale (Aurora)? (4) Latency requirements — sub-millisecond (DynamoDB+DAX) or millisecond (RDS)? (5) Analytics — use Redshift for OLAP. (6) Cost — serverless (DynamoDB) vs. provisioned (RDS). (7) Operational overhead — fully managed vs. more control.

Q30. What is a DB Subnet Group in Amazon RDS? Why is it required?

Ans: A DB Subnet Group is a collection of subnets in different Availability Zones within a VPC that you designate for your DB instances. It is required so that RDS knows which subnets to use when creating a DB instance in your VPC. For Multi-AZ, the subnet group must contain subnets in at least 2 AZs.

Q31. What is Aurora Backtrack? What problem does it solve?

Ans: Aurora Backtrack (MySQL only) allows you to rewind a DB cluster to a specific point in time without restoring from a backup. It solves the problem of accidental data modifications — instead of a lengthy restore process, you can quickly rewind to before the mistake occurred. It is near-instantaneous compared to traditional restores.

Q32. Compare DynamoDB, RDS, and Redshift in terms of data model, use case, and scalability.

Ans: DynamoDB: NoSQL (key-value/document), use case = real-time/serverless apps, scalability = automatic horizontal to any scale. RDS: Relational SQL, use case = OLTP/transactional web apps, scalability = vertical + limited read replicas. Redshift: Relational columnar (OLAP), use case = data warehouse/BI/analytics, scalability = horizontal MPP clusters up to petabyte scale.

8. Quick Reference Summary

8.1 Well-Architected Framework — Pillar Design Principles at a Glance

Pillar	Key Design Principles (Select)
Operational Excellence	Perform ops as code • Frequent small changes • Anticipate failure • Learn from failures
Security	Strong identity foundation • Enable traceability • Apply security at all layers • Protect data

Pillar	Key Design Principles (Select)
Reliability	Automatically recover • Test recovery • Scale horizontally • Stop guessing capacity
Performance Efficiency	Go global in minutes • Use serverless • Experiment more often
Cost Optimization	Adopt consumption model • Measure efficiency • Stop undifferentiated heavy lifting
Sustainability	Understand impact • Maximize utilization • Use managed services

8.2 AWS Database Services — Quick Selection Guide

Scenario / Keyword	Choose This Service
Traditional relational DB, OLTP, SQL	Amazon RDS (MySQL/PostgreSQL/SQL Server)
High performance relational, cloud-native, 5x MySQL speed	Amazon Aurora
Global relational DB across regions, <1 sec replication	Aurora Global Database
Auto-scale relational, intermittent use	Aurora Serverless
NoSQL, key-value, flexible schema, millions of TPS	Amazon DynamoDB
Microsecond latency for DynamoDB	DynamoDB + DAX
Multi-region active-active NoSQL	DynamoDB Global Tables
Event-driven on DB changes	DynamoDB Streams + Lambda
Data warehouse, analytics, BI, OLAP, petabyte scale	Amazon Redshift

Scenario / Keyword	Choose This Service
Query S3 data without loading into Redshift	Redshift Spectrum
Serverless analytics	Redshift Serverless
Near real-time Aurora to Redshift	Zero-ETL integration
Session management, gaming, IoT	DynamoDB
Complex SQL joins, reports, transactions	RDS / Aurora
BI tools (Tableau, QuickSight) integration	Amazon Redshift

Final Exam Tip: The most frequently tested concepts are: (1) Which pillar each design principle belongs to, (2) RDS Multi-AZ vs Read Replicas, (3) Aurora vs RDS differences, (4) When to use DynamoDB vs RDS, (5) What Redshift is used for (OLAP, not OLTP), and (6) Steps to launch an RDS instance. Always look for keywords in questions: 'disaster recovery' → Multi-AZ; 'read performance' → Read Replicas; 'petabyte analytics' → Redshift; 'flexible schema/NoSQL' → DynamoDB; '5x MySQL performance' → Aurora.

End of Study Notes — Best of Luck on Your Exam